Problem A. 01 Matrix

Rewrite the conditions using two-dimensional prefix sums. After this transformation, except for the parity at the bottom-right corner, every condition becomes a constraint saying that the parities on the bottom side and the right side of a rectangle must be equal or different.

Build a graph whose vertices are these parity variables. For every equality or inequality condition, connect the corresponding variables with an edge labeled by the required xor value. Each connected component has exactly two possible assignments once one vertex is fixed. Therefore, after checking consistency, multiply the answer by 2 for each connected component.

Problem B. Birds-of-Paradise' Card Game

If $3S \le W$, the trivial upper bound $64S$ is attainable. Assume henceforth $W < 3S$.

The official proof first shows that there is an optimal play sequence with no consecutive pattern `w,s,w,s`. If such a pattern appears, choose the earliest one. Depending on the preceding cards, replacing it by `w,w,s,s` or moving the last `w` to the right does not decrease the score, and this operation does not create an earlier violation. Repeating it terminates.

Now decompose a sequence into blocks. The relevant block types and scores are

block	w	s	ws	wss	$wsss$	wws	$wwss$	$wwsss$	$wwws$	$wwwss$	$wwwsss$
score	0	27	36	72	108	48	96	132	64	112	148

The only possible interactions between adjacent blocks can be ignored: two of them are improved by merging blocks from the beginning, and the remaining `ws`-to-`ws` interaction is excluded by the no-`w,s,w,s` normal form.

Next restrict the number of times small exceptional blocks are used. In an optimal solution it is enough to try

$$w \le 2, \quad ws \le 1, \quad wss \le 2, \quad wwss \le 2, \quad wws \le 1.$$

After enumerating these counts, the remaining blocks come from a small family. Further exchange arguments reduce the remaining choices to a constant number of linear systems, such as combinations of `s`, `wsss`, `wwsss`, `wwws`, `wwwss`, and `wwwsss`. Solving all these systems and taking the best value gives an $O(1)$ algorithm. A simpler implementation enumerates also `s`≤ 2 and solves the possible block combinations; it is still constant time, though with a larger constant.

Problem C. Don't be Clockwise

Choose m so that p_m is a point with maximum x-coordinate, and add an auxiliary point $p_{N+1} = (X_m, -10^9)$. Start the sequence with p_{N+1}, p_m, then order the remaining $N-1$ points.

For the third point, any remaining point can be chosen. If at every step we choose the point that makes the smallest turn from the current direction, then an angle-sort argument around the second point shows that all remaining points can still be chosen later. Repeating this greedy choice constructs a valid order. Finally, remove the auxiliary first point; the remaining sequence satisfies the required condition.

The straightforward implementation is $O(N^2)$, which is sufficient for the constraints.

Problem D. Except Ai

Because the operations are flexible, first ask when a constant sequence can be produced. Such a sequence exists exactly when there is a positive integer at most X that does not appear in the current segment.

Thus the problem reduces to: partition A into the minimum number of contiguous segments such that every segment is missing some number in $[1, X]$. Greedily scan from left to right. Extend the current segment as long as it still has a missing value; when extending would make all X values appear, cut before that position and start a new segment. Cutting earlier cannot help, because shorter segments are never harder to make valid.

Maintain the set or count of distinct values in $[1, X]$ in the current segment to implement the scan in $O(N)$.

Problem E. Find "rururutata"

The problem can be solved by enumerating runs, but the intended solution computes two auxiliary arrays:

- for every ending position, the minimum length of a 3-rep sequence ending there;
- for every starting position, the minimum length of a 2-rep sequence starting there.

Here an i-rep sequence is a nonempty block repeated i times.

For a 3-rep ending at a fixed position, the naive method tries all block lengths and checks equality by hashing, giving $O(N^2)$. Improve this by grouping block lengths by powers of two. When the block length lies in $[2^t, 2^{t+1})$, only candidates whose last 2^t elements match need to be tested.

Still, by itself this can be quadratic. Add one more pruning rule: once a 3-rep has been found for an ending position, stop exploring that ending position. If candidates of the same scale overlap by at least $2/3$, then the overlapped part already forms a 3-rep, so this situation cannot occur many times. Hence each scale performs only $O(N)$ tests, and all scales together take $O(N \log N)$. Hashes for blocks of length 2^t are built from those of length 2^{t-1}, and equality checks are constant time. The 2-rep array is computed symmetrically.

Problem F. Increase Decrease

Let A_i and B_i be the prescribed LIS and LDS values. Necessary conditions are:

- both A and B are nondecreasing;
- $A_1 = B_1 = 1$;
- $(A_{i+1} - A_i) + (B_{i+1} - B_i) \leq 1$.

Additionally, define X_i as the longest increasing subsequence length ending at C_i, and Y_i as the longest decreasing subsequence length ending at C_i. Two indices cannot have the same pair (X_i, Y_i): if $C_i < C_j$, the increasing value contradicts equality, and if $C_i > C_j$, the decreasing value does. Therefore, when embedding elements into grid cells (X_i, Y_i), if a cell (x, y) is occupied, the cells $(x-1, y)$ and $(x, y-1)$, when meaningful, must also be occupied. This implies $i \leq X_i Y_i$.

These conditions are also sufficient. Construct the grid while processing A, B: when one of the two values increases, extend the corresponding boundary of the grid. For restoration, keep the sequences added to the upper and lower sides of each cell expansion, and fill cells greedily in increasing value order. This yields an $O(N)$ construction.

Problem G. Make T

The official article notes that the written solution is incorrect for the original statement; it becomes correct if every alien is required to hold hands with at least two other aliens. Under that strengthened condition, the following reduction is valid.

Call an alien good if it holds hands with exactly three aliens. Corner aliens have only two possible neighbors, so they can never be good. Boundary non-corner aliens have exactly three neighbors, so in an optimal solution they may be assumed to hold all possible hands. The same exchange argument applies to all four sides: adding a missing boundary edge can only make the boundary alien good, and can hurt at most one interior alien.

Equivalently, start from the grid graph with all possible edges present, and remove only edges between pairs that were originally not connected. Since an operation that makes an already-good alien bad cannot improve the answer, every non-corner alien of final degree at most two can be excluded. The remaining choice becomes a maximum matching problem on a bipartite graph with $O(N^2)$ vertices and edges. Hopcroft–Karp solves it in $O(N^3)$.

Problem H. OR Preference

For small subtasks, brute force all merge orders or use interval DP:

$$dp[l][r][a] = \text{maximum number of OR operations needed to make value } a$$

from the interval $A[l, r)$, with impossible states set to $-\infty$. This gives a direct but expensive solution.

For the full solution, use a swap argument. Suppose an AND operation merges values a, b, and an OR operation was used inside the construction of a. Swapping this OR operation with the current AND operation preserves the possibility of ending with 0: any bit that was already 1 cannot become harmful, because the final operation in the swapped part is OR. Repeating this argument shows that there is an optimal sequence in which all OR operations are done before all AND operations.

Then a prefix DP suffices:

$$dp[r][a] = \text{maximum OR count to make value } a \text{ from } A[0, r).$$

Let $V = \max A$. Naively this DP costs $O(NV^2)$. For a fixed start i, the values of

$$A_i \ \& \ A_{i+1} \ \& \cdots \& \ A_j$$

change only $O(\log V)$ times. Hence transitions can be grouped and updated by an imos-style method in $O(NV \log V)$. The official final optimization uses the idea behind fast zeta transform: as j increases, each bit of the AND value changes monotonically, so all transitions for one i can be processed in about $O(N + V)$, removing the logarithm in the intended full solution.

Problem I. Penguin Flicker

Compress each original length-one location containing a penguin into a length-zero point. The contribution of the interval to the left of the first penguin, between consecutive penguins, and to the right of the last penguin depends only on N and the interval length, so these values can be precomputed.

A direct $O(N^2)$ DP table seems natural, but its transitions form cycles. Instead, write the recurrence as a system of linear equations. Each variable depends only on itself and its two neighboring variables. Because the matrix is tridiagonal, two forward/backward elimination passes are enough, so all needed values can be computed in $O(N^2)$.

Finally, undo the length compression carefully. The endpoints may shift; fix the number of penguins falling to the left and correct the boundary contribution. The official note mentions that the time limit is tight, so computing only one triangular half of the DP table and reducing divisions in modular arithmetic are useful implementation optimizations.

Problem J. Set Sequence

Define

$$F(x_1, \ldots, x_N) = \sum_{T \subseteq \{1,\ldots,N\}} \prod_{i \in T} x_i = \prod_{i=1}^{N} (1 + x_i).$$

The number of ways for a fixed $|T| = K$ can be represented as

$$\left[\prod_{i=1}^{N} x_i^{A_i} \right] (F - 1)^K.$$

Expanding by powers of F, the coefficient of F^t in the total sum is

$$\sum_{K=t}^{\sum A_i} (-1)^{K-t} \binom{K}{t},$$

while

$$\left[\prod_i x_i^{A_i} \right] F^t = \prod_i \binom{t}{A_i}.$$

Thus it remains to enumerate these two quantities over t. For $t \geq p$, the second product is 0 modulo p, so only $t < p$ matters. The first value is easy to compute; the second is a multipoint evaluation problem. Taking a primitive root modulo p, the evaluation points form a geometric progression, so the chirp z-transform removes one logarithmic factor. The complexity is $O(N \log^2 N + p \log p)$.

Problem K. Talk Event

First count selections that take exactly X units of time. Let t_i be the number of selected people who bought i tickets. We need

$$0 \le t_i \le T_i, \quad \sum_{i=1}^{4} t_i = N, \quad \sum_{i=1}^{4} i t_i = X.$$

Ignore upper bounds temporarily and remove invalid cases by inclusion-exclusion. After rearranging,

$$t_2 + t_3 + t_4 \le N, \qquad t_2 + 2t_3 + 3t_4 = X - N.$$

Set

$$x_1 = t_4, \quad x_2 = t_3 + t_4, \quad x_3 = t_2 + t_3 + t_4.$$

Then

$$x_1 \le x_2 \le x_3 \le N, \qquad x_1 + x_2 + x_3 = X - N.$$

Drop the ordering first and count triples with $\max x_i \le N$ by inclusion-exclusion and binomial coefficients. Usually divide by 6; correct separately for equal coordinates. This gives the exact-X count in $O(1)$.

For the original condition, if $t_1 \ne T_1$, the total time must be exactly X; otherwise reduce the instance and allow smaller time. The same reasoning applies to t_2, t_3, t_4, so only four exact-time instances need to be evaluated. Be careful with the case where everyone can be selected and all are selected: then time below $X - 3$ may also be allowed.

Problem L. Unique Sheet

If some integer a with $1 \le a \le (N-K)^2$ never appears in the matrix, the answer is immediately 0. Assume every such integer appears.

The brute-force search over deleted rows and columns costs $\binom{N}{K}^2$, which is impossible. The first pruning idea is that a row or column containing a globally unique needed value cannot be deleted. For large N, this reduces the candidate deleted rows and columns to at most about 10 each when $K = 5$, so enumeration becomes feasible.

For smaller N, apply a second pruning after fixing the deleted rows. Let the remaining rectangular matrix be A'. Any value that appears exactly once in A' must not be deleted by the column choice. For example, when $N = 17, K = 5$, the remaining matrix has $12 \cdot 17 = 204$ entries but only $(N-K)^2 = 144$ needed values, so at least 60 entries force restrictions; the number of candidate columns becomes small enough.

Using the first idea for large instances and the second for small instances, enumerate only the surviving row/column sets and check each in $O(N)$. Extra constant-factor pruning, such as handling values appearing exactly twice, may be useful in tight implementations.

Problem M. Up-Down Sequence

For a permutation P, call (i, j, k) an increasing triple if $P_i < P_j < P_k$, and a decreasing triple if $P_i > P_j > P_k$. We need equal counts.

One construction pairs symmetric positions. Assign P_k and P_{N-k} the two values $2k-1$ and $2k$. Most increasing triples pair with decreasing triples under $(i, j, k) \mapsto (N-k, N-j, N-i)$; only triples involving symmetric indices need attention. Choosing $P_k = 2k-1$ increases $U - D$ by $k-1$, while choosing $P_k = 2k$ decreases it by $k-1$.

Thus, for $N = 2M$, it is enough to partition $\{0, 1, \ldots, M-1\}$ into two sets of equal sum. This is possible iff $M \equiv 0, 1 \pmod 4$, and explicit patterns give the partition. This solves $N \equiv 0, 2 \pmod 8$.

For the other residues, insert a small middle block Q between the two halves of the above construction:

- if $N \equiv 1, 3 \pmod 8$, use $Q = (N)$;
- if $N \equiv 4, 6 \pmod 8$, use $Q = (N-2, N, N-3, N-1)$;
- if $N \equiv 5, 7 \pmod 8$, use $Q = (N-3, N, N-2, N-4, N-1)$.

The inserted block has balanced increasing/decreasing triple contribution and the required inversion contribution. Hence every N has an $O(N)$ construction. The official alternative editorial gives another $O(N)$ construction by concatenating four large monotone blocks for $N = 4M$ and modifying it for the other residues.

Problem N. Complete Set

For buyers who purchase a complete set, there is no decision to make. Only buyers who purchase a single item matter.

The greedy strategy is to sell the item with the largest remaining stock. The total stock count changes in the same way regardless of which single item is sold, as long as the process does not fail. What matters is to keep the minimum stock among item types as large as possible, because that minimum controls how many complete sets can still be sold. Selling from a currently largest stock never decreases this minimum unnecessarily.

Selling from a minimum stock would be dangerous: the input can be arranged so that complete-set buyers continue forever after such a move, so that operation must be avoided. Implement the greedy strategy with a priority queue or ordered set.

Problem O. Giraffe? Zebra?

For giraffe strings, the count follows directly from the definition.

For zebra strings, split the string at positions where two equal adjacent characters meet. Inside one resulting block, the count is computed in the same way as for giraffe strings. For interval queries, handle only the two boundary blocks explicitly; all full blocks strictly inside the query interval are accumulated by prefix sums.

An okapi string is both a giraffe string and a zebra string. Count these with the same block decomposition as for zebra strings, adding the extra giraffe constraint locally at the two boundaries and by prefix sums in the middle.

Problem P. Simple Tree Paint Game

Let d be the distance between the starting vertices a and b. If d is odd, Alice wins; if d is even, Bob wins.

When d is odd, Alice can always move along the path toward Bob until she reaches the same vertex as Bob. The parity of the distance guarantees that she can do this at the right turn. After that, Alice mirrors Bob's moves. Whatever Bob does, Alice can maintain the mirror position and wins.

For even d, the same argument with the roles swapped gives Bob a winning strategy. Thus the solution is to compute the distance parity by BFS/DFS or LCA preprocessing, in $O(N)$ for one game.

Problem Q. Thorny Path

There are N possible paths. Call a path "move i" if the downward step is made at the i-th move. Let S_i be the total thorniness of move i before any cutting.

Cutting cell $(1, j)$ decreases $S_j, S_{j+1}, \ldots, S_N$ by 1. Cutting cell $(2, j)$ decreases $S_1, S_2, \ldots, S_j$ by 1. Therefore, if cell $(1, j)$ is going to be cut while a positive-thorniness cell $(1, j-1)$ remains, cutting the earlier cell first is never worse; it decreases all path sums affected by $(1, j)$ and possibly more. A symmetric statement holds on the second row.

Hence there is an optimal strategy satisfying:

- before cutting $(1, j)$, all cells $(1, 1), \ldots, (1, j-1)$ have thorniness zero;
- before cutting $(2, j)$, all cells $(2, j+1), \ldots, (2, N)$ have thorniness zero.

This justifies the greedy algorithm. For $i = 1, 2, \ldots, N$, while the target T_i is smaller than the current S_i, cut the leftmost positive cell among $(1, 1), \ldots, (1, i)$ if it exists; then cut positive cells on row 2 from right to left down to column i. Maintain the current leftmost/rightmost positive cells and the affected range values with a segment tree.

Problem R. Subtree with Lower Limit

Let par_u be the parent of vertex u for $u = 2, 3, \ldots, N$. Clearly $par_u < u$ is necessary.

Conversely, any sequence satisfying $par_u < u$ defines exactly one rooted tree: connect each u to par_u. Repeatedly moving from u to par_u strictly decreases the label, so every vertex eventually reaches the root. Thus the graph is connected; with $N-1$ edges, it is a tree. Different parent sequences produce different trees.

Therefore the answer is simply the number of choices

$$par_2 \in \{1\}, \quad par_3 \in \{1,2\}, \quad \ldots, \quad par_N \in \{1,\ldots,N-1\},$$

which is

$$1 \cdot 2 \cdots (N-1) = (N-1)!.$$